BABEŞ-BOLYAI UNIVERSITY CLUJ-NAPOCA
FACULTY OF MATHEMATICS AND COMPUTER SCIENCE
SPECIALIZATION COMPUTER SCIENCE

DIPLOMA THESIS

NEST: A Localized Web Application for Enhancing Residential Community Engagement

Supervisor
Dr. Bocicor Maria Iuliana

Author
Bălă Bogdan Andrei

2026

Abstract

In high-density urban residential blocks, physical proximity is paradoxically accompanied by deep social isolation and a lack of community trust. Neighbors share the same building structure for years without developing meaningful social capital, making simple daily challenges, such as coordinating local events, borrowing household tools, or temporary parking space allocation, inefficient and friction-filled. This thesis presents NEST, a secure, responsive, and highly reactive web application designed specifically to convert localized digital coordination into physical neighborhood cooperation.

NEST implements a decoupled Client-Server architecture. The presentation layer is built upon Angular 21, utilizing standalone components and reactive Signal APIs to achieve fine-grained DOM rendering. State management is structured around a global NgRx store encapsulated within the Facade pattern. The backend server is built using Node.js, Express, and TypeScript, employing a multi-layered middleware chain for JWT authorization, role-based access control, and rate limiting. Data persistence is managed via an SQLite database, abstracted through the Prisma ORM.

The core contribution of this work includes the design and implementation of eight functional modules, featuring a dynamic CSS design token system with over 600 custom properties for seamless light/dark theming, and an automated conflict-resolution engine that prevents double-booking of parking slots and shared tools. Systematic testing and verification confirm zero compilation errors, full cross-browser responsiveness, high resilience against unauthorized API actions, and excellent usability. The results validate NEST as a secure, high-performance, and scalable digital hub that actively restores social capital and mutual aid within residential communities.

This work is the result of my own activity. I have neither given nor received unauthorized assistance on this work.

Contents

Chapter 1

Project Context and Motivation

1.1 Urban Isolation in Apartment Communities

In modern metropolitan centers, high-density residential developments have paradoxically fostered social isolation. Neighbors live in physical proximity, separated only by thin concrete partitions, yet they often share no meaningful social capital or interpersonal trust. This social fragmentation is part of a broader decline in localized civic participation and community cohesion, a phenomenon well-documented by sociologists such as Robert Putnam in his seminal work on the erosion of social capital [1].

The consequences of this disconnect are not merely social, but highly practical. Without active communication channels, residents miss opportunities for mutual aid, shared resource allocation, building governance coordination, and safety collaboration. Simple daily challenges, such as finding a temporary parking spot, borrowing a household tool, or planning building-wide maintenance, become friction-filled tasks because there is no trusted hyper-local communication platform.

1.2 Local Digital Networks and Mutual Aid

To bridge this social gap, urban informatics researchers have investigated how digital communication networks can be designed to support physical communities. Keith Hampton and Barry Wellman's research on localized networks demonstrates that web-based platforms, when designed specifically for geographically bounded neighborhoods, act as catalysts for real-world social interaction rather than replacement channels [2].

Hyper-local systems such as Nextdoor [3] or building-wide WhatsApp groups are commonly deployed, yet they exhibit significant limitations:

- **Commercial Noise and Geographic Limitations:** Global platforms like Nextdoor

are heavily commercialized and suffer from strict regional restrictions, making them unavailable in Romania and many other parts of the world. Additionally, they introduce commercial noise and pose privacy concerns that undermine local resident trust [4].

- **Informal Group Chat Chaos:** Instant messaging channels, such as WhatsApp or Telegram, lack structured categorization. Announcements, item lending requests, and event coordinates quickly become lost in continuous text streams, causing user fatigue.
- **Lack of Verification:** General social networks do not provide building-level verification. Anyone can join, making transactions like tool sharing or parking allocation risky.

NEST addresses these limitations by providing a structured, building-specific digital space. It is designed to turn online coordination into real-world cooperation.

1.3 NEST Project Overview

NEST is a hyper-local web application designed exclusively for residents of a single apartment building block. It provides a secure, structured space where residents can build trust, coordinate mutual aid, and organize community initiatives.

Rather than trying to replicate a broad social network, the system focuses on structured utility modules:

- **Social Hub:** Houses the chronological Community Feed where verified residents share general announcements and building news.
- **Collaborative Consumption:** Operates the Shared Shed, enabling safe peer-to-peer household item lending and borrowing.
- **Spatial Optimization:** Integrates the Parking Sharing system, helping residents coordinate temporary parking slot access.
- **Physical Organizing:** Supports the Events module to facilitate real-world neighborhood meetings and gatherings.

My contribution to the NEST project is the frontend design and implementation, utilizing Angular 21, reactive Signals, and NgRx state management, alongside the integration of robust backend services, JWT verification middleware, database modeling with Prisma ORM, and the custom color CSS design token system.

1.4 Project Objectives

The NEST platform was developed with five core objectives:

1. **Objective 1 (O1): Building Verification Security** – Design and implement a secure, multi-stage registration system requiring administrative approval before granting residents access to building data.
2. **Objective 2 (O2): Cohesive Design System** – Create a unified CSS design token system that supports responsive layouts and instant light/dark theme transitions across all modules.
3. **Objective 3 (O3): Collaborative Consumption (Shared Shed)** – Implement a peer-to-peer tool-sharing catalog with conflict prevention and transaction history tracking.
4. **Objective 4 (O4): Spatial Sharing (Parking)** – Build a parking spot sharing system with auto-rejection logic for overlapping requests.
5. **Objective 5 (O5): High-Performance Reactive Architecture** – Structure state management using the NgRx Facade pattern and Angular Signals to ensure fast initial loads and fine-grained DOM rendering.

1.5 Original Contributions

This thesis documents several technical contributions in localized web engineering:

- **Adapted Facade-Signals Reactive Architecture:** Adapted the established NgRx Facade design pattern and integrated it with Angular's modern Signal-based reactivity system to build a state management architecture tailored to NEST's multi-module structure, isolating store mechanics from UI components while leveraging fine-grained template reactivity.
- **Dynamic CSS Theme Engine:** Constructed a design token system containing over 600 custom properties, achieving instant light-to-dark theme transitions without DOM re-renders.
- **Secure Multi-Stage Verification Flow:** Developed a routing and security guard pipeline that limits unverified users to onboarding views until approved by the building administrator.
- **Conflict-Resilient Rental & Lending Logic:** Implemented backend transaction checks using Prisma ORM to prevent double-booking of parking slots and shared tools.

1.6 Thesis Structure

The remainder of this thesis is organized as follows. Chapter 2 reviews the theoretical background and state of the art, covering single-page application paradigms, reactive state management, CSS architecture, web security, and related community platforms. Chapter 3 presents the software analysis and design phase, detailing the requirements specification, system architecture, database modeling, and REST API contract definitions. Chapter 4 describes the implementation details, including the codebase organization, component structures, the NgRx Facade integration, core module logic, and security middleware. Chapter 5 discusses the testing and validation strategies, presenting build verification outputs, cross-browser compatibility checks, and usability considerations. Finally, Chapter 6 summarizes the project outcomes, identifies current engineering limitations, and proposes directions for future development.

1.7 Declaration on the Use of AI Tools

In compliance with the academic integrity guidelines of Babeş-Bolyai University (Faculty of Mathematics and Computer Science), I declare that generative Artificial Intelligence tools, specifically large language models, were utilized during the development and writing phases of this thesis.

AI assistance was specifically applied for:

- **Text Rephrasing:** Assisting in rewriting drafts of the thesis text to improve syntactic flow, clarity, and grammatical cohesion.
- **Technical Expression Enhancement:** Refining descriptions of complex software engineering concepts and architectural styles to align with professional, academic phrasing standards.
- **UI Design Generation:** The AI assistant integrated within Figma was used to help generate user interface design mockups based on descriptive prompts provided by the author. All generated designs were subsequently reviewed, refined, and adapted to meet the specific requirements of the NEST application.

All code, architectural designs, database schemas, and written analyses were reviewed, compiled, and verified by the author to ensure technical accuracy and academic integrity.

Chapter 2

Theoretical Background and State of the Art

This chapter surveys the foundational concepts and existing body of knowledge upon which the NEST application is constructed. Rather than merely listing tools, the aim here is to provide a reasoned synthesis of the architectural paradigms, design patterns, and engineering principles that shaped the technical decisions made throughout this project. Each section introduces a concept from the literature, discusses its relevance, and explains why a particular approach was selected over its alternatives.

2.1 Web Application Architecture Paradigms

The way a web application is structured at the highest level determines its performance characteristics, maintainability, and user experience. Three dominant paradigms have emerged over the past two decades.

Multi-Page Applications (MPAs) represent the earliest model, where each user interaction triggers a full page reload from the server. While simple to reason about, this approach results in noticeable latency and a disjointed user experience, because the entire HTML document must be re-fetched for every navigation action [5].

Single-Page Applications (SPAs) emerged as a response. In this model, the browser loads a single HTML shell and then dynamically rewrites the page content using JavaScript [6]. Navigation occurs client-side through the History API, and data is fetched asynchronously from the server through API calls [7]. The perceived responsiveness is substantially better, since only data payloads travel over the network rather than complete rendered pages.

Server-Side Rendering (SSR) and hybrid approaches attempt to combine the SEO advantages of MPAs with the interactivity of SPAs, pre-rendering the initial

HTML on the server and then "hydrating" it with client-side JavaScript.

For a community platform like NEST, the SPA approach was chosen. The primary audience, apartment building residents, will use the application for short, frequent sessions (checking the feed, browsing available tools, RSVPing to an event). These interactions demand near-instant visual feedback, which a full-page reload model cannot deliver. Furthermore, since NEST operates behind an authentication wall and is not indexed by search engines, the SEO trade-off of a pure SPA is irrelevant.

The client-server communication follows the Representational State Transfer (REST) architectural style, as defined in Fielding's doctoral dissertation [8]. REST imposes a set of constraints, statelessness, uniform interface, layered system, that result in predictable, resource-oriented APIs. Each endpoint maps to a domain entity (e.g., `/api/v1/feed`, `/api/v1/events`), and standard HTTP verbs (`GET`, `POST`, `PUT`, `DELETE`) describe the operations upon them.

2.2 Component-Based Frontend Development

Modern frontend development has converged on the idea of decomposing user interfaces into independent, reusable *components*. A component encapsulates its own template (structure), styles (appearance), and logic (behavior) in a single cohesive unit. This mirrors the broader software engineering principle of high cohesion and low coupling [9].

Three frameworks dominate this space: React [10] (maintained by Meta), Vue.js [11], and Angular [7] (maintained by Google). Table 2.1 summarizes their key differences.

Criterion	**React**	**Vue.js**	**Angular**
Architecture	Library (view layer only)	Progressive framework	Full opinionated framework
Language	JSX (JavaScript)	SFCs (JavaScript or TS)	TypeScript (mandatory)
State model	External (Redux, Zustand)	Reactive refs, Pinia	Built-in DI, NgRx optional
Routing	Third-party (React Router)	Vue Router (official)	Built-in Angular Router
Form handling	Third-party (Formik, etc.)	Third-party	Built-in Reactive Forms
Learning curve	Moderate	Low–Moderate	Steep

Table 2.1: High-level comparison of dominant frontend frameworks

Angular was selected for NEST for several concrete reasons. First, its mandatory use of TypeScript provides compile-time type checking across the entire fron-

tend, catching errors before they reach the browser. Second, Angular ships with integrated solutions for routing, forms, HTTP communication, and dependency injection, which eliminates the "decision fatigue" of assembling a bespoke tool stack. Third, the framework's opinionated project structure, with conventions for modules, services, guards, interceptors, and pipes, enforces a level of architectural discipline that is particularly valuable for a project with multiple feature domains [7].

2.2.1 Standalone Components and the Modern Angular Model

Historically, Angular organized components into `NgModules`, which served as compilation units and dependency containers. Starting with Angular 14 and fully maturing in subsequent versions, the framework introduced *standalone components*. A standalone component declares its own imports directly in the `@Component` decorator, removing the need for an intermediate module class [7].

This simplification has two consequences. At the technical level, it enables more granular tree-shaking: the compiler can exclude unused components from the production bundle because dependencies are declared at the component level rather than at the module level. At the organizational level, it makes the codebase more readable, because the reader can see exactly what a component depends on by inspecting its decorator metadata rather than tracing imports through a module hierarchy.

The NEST application adopts standalone components throughout, with zero `NgModule` declarations.

2.3 Reactive State Management

As web applications grow in complexity, managing the data that flows between components becomes a significant engineering challenge. Consider a community platform where a single user action, such as approving a parking request, must simultaneously update a notification counter, a list view, and a detail panel. If each component manages its own local copy of the relevant data, inconsistencies and race conditions quickly arise.

2.3.1 The Redux Architecture

The Redux architecture, introduced by Abramov and Clark [12], addresses this problem by centralizing all application state in a single, immutable store. State transitions occur exclusively through *actions* (plain objects describing what happened) and *reducers* (pure functions that compute the next state). Side effects, such as HTTP

requests, are handled outside the reducer through middleware or, in the Angular ecosystem, through *effects*.

This architecture enforces a strict unidirectional data flow:

$$\textit{Component} \xrightarrow{\text{dispatches}} \textit{Action} \xrightarrow{\text{processed by}} \textit{Reducer} \rightarrow \textit{New State} \xrightarrow{\text{selected by}} \textit{Component}$$

NgRx is the Angular-specific implementation of this pattern [13]. It integrates with Angular's dependency injection system and provides additional constructs such as *selectors* (memoized state projections) and *effects* (RxJS-based side-effect handlers).

2.3.2 Angular Signals and Fine-Grained Reactivity

Angular traditionally relied on Zone.js for change detection: after any asynchronous event (a click, a timer, an HTTP response), the framework would walk the entire component tree to check for differences. While functional, this approach performs unnecessary work in large applications.

Angular Signals, formally proposed in 2023 [14], introduce a fundamentally different reactivity model. A `signal` is a reactive primitive that tracks its own dependencies. When a signal's value changes, only the components that read that specific signal are notified, there is no tree-wide check.

Signals come in several forms:

- `signal(value)` , a writable reactive container.
- `computed(() => ...)` , a derived value that automatically recomputes when its dependencies change.
- `input()` and `output()` , signal-based replacements for the older `@Input()` and `@Output()` decorators, used for parent-child component communication.
- `effect(() => ...)` , a side-effect that runs whenever the signals it reads are updated.

In the NEST application, both paradigms coexist. The global, cross-cutting state (user session, feed posts, event lists) is managed through NgRx stores, while local component concerns and derived computations use signals. This hybrid approach is possible because NgRx itself exposes its store as a signal through `selectSignal()`.

2.3.3 The Facade Pattern for Store Access

A recurring problem in NgRx applications is that components become tightly coupled to the store's internal structure. If a selector or action name changes, every component that references it must be updated. The *Facade pattern* [15] solves this by

introducing an intermediary service that encapsulates all store interactions behind a clean, domain-specific API.

For example, rather than having a component dispatch `AuthActions.login({email, password})` directly, it calls `authFacade.login(email, password)`. The Facade translates this into the appropriate dispatch, and also exposes selectors as signals. This decoupling means that the store's internal structure can be refactored without touching any component code.

2.4 Backend Architecture: Layered Services

The backend of a web application can be organized in many ways, but a layered architecture, where each layer has a well-defined responsibility and communicates only with its adjacent layers, is widely recommended for maintainability [16].

The layers adopted in NEST are:

1. **Routes**: Define the URL endpoints and attach middleware functions.
2. **Controllers**: Parse the incoming HTTP request, delegate business logic to the service layer, and format the HTTP response.
3. **Services**: Contain the core business logic and interact with the database.
4. **Data access (Prisma)**: Translate service-level queries into SQL statements and return typed results.

This separation ensures that, for instance, the logic for approving a parking application can be reused regardless of whether the trigger is an HTTP request, a scheduled task, or a future WebSocket event.

Node.js was chosen as the runtime because it shares the same language (JavaScript/TypeScript) as the frontend. This eliminates the cognitive overhead of context-switching between languages and enables shared type definitions and validation schemas between client and server. Express, the most widely adopted Node.js web framework, provides a minimal but flexible middleware pipeline [17].

2.5 Authentication and Authorization

Securing a community platform is not merely a technical concern but a social one. Residents must trust that only verified neighbors can view their posts, borrow their belongings, and access their contact information.

2.5.1 JSON Web Tokens

Token-based authentication, specifically through JSON Web Tokens (JWTs), has become the standard approach for stateless API authentication [18]. A JWT consists

of three base64-encoded segments: a header (specifying the signing algorithm), a payload (containing claims such as user ID and role), and a signature (ensuring the token has not been tampered with).

The key advantage of JWTs over traditional session cookies is that the server does not need to maintain session state. Each request carries its own proof of identity, which the server can verify by checking the signature against a secret key. This property aligns well with the stateless constraint of REST [8].

2.5.2 Password Hashing with bcrypt

Storing passwords in plaintext, or even with simple hash functions like MD5, is a well-known vulnerability. The bcrypt algorithm, introduced by Provos and Mazières [19], is specifically designed for password hashing. It incorporates a configurable *cost factor* that controls the computational expense of hashing. As hardware improves, the cost factor can be increased to maintain resistance against brute-force attacks.

2.5.3 Role-Based Access Control

Authorization in NEST follows a Role-Based Access Control (RBAC) model. Each user is assigned a role, currently `RESIDENT` or `ADMIN`, and middleware functions enforce role constraints at the route level. A multi-step verification flow adds an additional layer: a newly registered user cannot access the community until an administrator of their apartment block explicitly approves their account. This mirrors the real-world process of verifying residency.

2.6 Object-Relational Mapping and Database Design

Relational databases remain the dominant choice for structured application data, but writing raw SQL queries in application code introduces maintenance burdens and security risks (SQL injection). Object-Relational Mapping (ORM) libraries bridge the gap between the relational model and the object-oriented or type-based models used in application code [20].

Prisma, the ORM selected for NEST, takes an unusual approach: rather than inferring the schema from code annotations (as Hibernate or TypeORM do), Prisma defines the schema in a dedicated `schema.prisma` file and generates a fully typed client library from it [21]. This means that every database query in the application is type-checked at compile time. If a column is renamed or a relation changes, the TypeScript compiler immediately flags all affected call sites.

The generated Prisma Client also handles relation loading (eager or lazy), pagination, and transaction management, keeping the service-layer code focused on business logic rather than query construction.

2.7 CSS Architecture and Theming

A common oversight in frontend development is treating stylesheets as an afterthought. In a multi-module application with dozens of components, ad-hoc CSS quickly becomes unmaintainable: colors, spacing values, and font sizes are duplicated across files, and any visual change requires a global search-and-replace.

CSS custom properties (also called CSS variables) address this problem at the language level [22]. A custom property is declared on a parent element, typically `:root`, and inherited by all descendant elements. By centralizing all visual tokens (colors, spacing scales, shadow definitions) in a single file and referencing them via `var(--token-name)` throughout the application, changes can be made in one place and propagated instantly.

This mechanism also enables theming. By redefining the same set of custom properties under a different selector (e.g., `[data-theme="dark"]`), the entire application's visual appearance can be switched by toggling a single attribute on the root HTML element. No component-level changes are required. This approach was adopted for the NEST application's light and dark theme system.

2.8 Related Work and Existing Platforms

Several existing platforms address neighborhood communication. *Nextdoor* is the most prominent, serving millions of users across neighborhoods worldwide. However, research has identified significant tensions in its design, including conflicts between commercial interests and community needs, as well as difficulties in scaling across neighborhoods with different norms [4].

Other approaches include building-specific WhatsApp groups and Homeowners Association (HOA) management software. WhatsApp groups lack structure, there is no way to categorize a message as an event versus a lending offer, and they become noisy at scale. HOA software, conversely, tends to focus on administrative functions (dues collection, maintenance tickets) rather than fostering social interaction.

NEST differentiates itself by focusing specifically on a single apartment building and structuring interactions around distinct community functions: a shared feed for announcements, an event planner with RSVP tracking, a mutual-aid tool-lending

module, and a parking spot sharing system. The scope is deliberately narrow to maintain relevance and trust. As Hampton and Wellman observed, digital networks are most effective at strengthening community ties when they are designed for a specific, bounded group of people [2].

Chapter 3

Software Analysis and Design

To transform the theoretical objective of fostering hyper-local resident engagement and mutual aid into a robust, scalable, and secure software solution, a rigorous software analysis and design phase is required. This chapter details the architectural patterns, structural design decisions, data models, and interface contracts that govern the NEST application, mapping the software lifecycle from specifications to architectural blue-prints.

3.1 System Architecture Overview

The NEST platform is built upon a modern Client-Server architectural style, employing a decoupled Single Page Application (SPA) frontend and a RESTful stateless backend API server. This separation of concerns guarantees that the presentation logic, business core, and data persistence layers remain highly modular, enhancing horizontal scalability and maintainability [16].

The complete data and control flow is organized into three distinct tiers:

1. **Presentation Tier (Frontend Client):** Built using the Angular framework (v21), this layer is compiled into static assets and executed entirely within the client's web browser. It is responsible for rendering user interfaces, managing client-side view state via reactive signals, and routing. It communicates asynchronously with the backend server via HTTP requests using JSON payloads.

2. **Application and Services Tier (Backend Server):** Built upon Node.js and the Express.js framework using TypeScript, this layer acts as the API Gateway and the central orchestrator of business logic. It handles client routing, evaluates security policies through a chain of modular middleware (including JWT validation, rate limiting, and Role-Based Access Control), validates request schemas, delegates core logic to specialized service objects, and communicates with the persistence layer.

3. **Persistence and Data Tier (Database and ORM):** Relational data storage is handled by an SQLite database, abstracted through the Prisma Object-Relational Mapping (ORM) engine. Prisma delivers compile-time type safety for database operations, acts as the query builder, and handles schema migrations.

The global client-server interaction model is depicted in Figure 3.1. When a client issues a request (e.g., submitting a post or requesting an item from the Shared Shed), the HTTP message traverses the network, is processed by the Express.js router, passes through authentication and authorization middleware chains, and triggers the target controller. The controller invokes the appropriate service layer, which issues type-safe Prisma queries to update the database, returning a JSON response that the Angular frontend consumes to reactively update the client-side state.

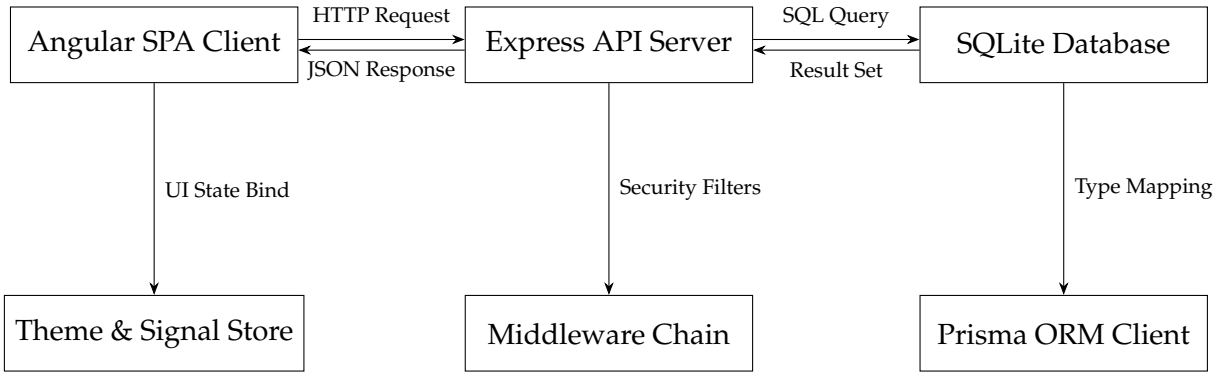

Figure 3.1: System architecture overview and multi-tier data flow of the NEST application.

3.2 Requirements Specification

The analysis phase of the software lifecycle requires defining functional and non-functional requirements to align implementation with resident needs and operational safety.

3.2.1 Functional Requirements

NEST is designed to consolidate isolated neighborhood interactions into 8 distinct functional modules. Table 3.1 maps these core modules to their detailed system capabilities and actor permissions.

3.2.2 Non-Functional Requirements

Non-functional requirements dictate the performance boundaries, operational safety parameters, and software quality characteristics of the platform [9].

Module	Feature Requirement	Functional Description and Permissions
Authen-tication	User Registration, Login & Building Join Request	Captures user profile fields and hashes passwords. Restricts standard application access until user joins a physical building block via unique block code, subject to Admin approval.
Community Feed	Chronological Post Feed & Media Attachment	Verified building residents can publish text announcements, attach images, and scroll through building updates. Supports real-time rendering.
Shared Shed	Peer-to-Peer Tool Sharing & Borrowing	Residents can list household appliances or tools. Other verified neighbors can request to borrow items, transitioning item states from available to pending and borrowed.
Events	Local Activity Planning & RSVP Tracking	Residents can schedule neighborhood meetings, clean-up activities, or parties. Limits participant count and tracks attendee lists reactively.
Parking Sharing	Parking Space Listing & Short-Term Booking	Slot owners register slots and announce temporary availability. Neighbors browse available parking intervals and apply to reserve the spot.
Profile	Resident Details & Activity Summaries	Displays contact info (with visibility settings), profile/cover images, biography, and history of active listings and events.
Settings	Reactive Preference & Theme Customization	Allows instant light/dark theme switching, individual toggle controls for notification channels, and phone number privacy settings.
Admin Panel	Block Moderation & Access Approval	Enables building block administrators to approve or deny resident join requests, moderate posts, and manage block metadata.

Table 3.1: Core Functional Requirements of NEST by Feature Module

- **Security and Privacy:** Confidentiality is a primary concern. All HTTP client-server communication must use secure protocols. Sensitive fields such as passwords must be hashed using the bcrypt algorithm with a workload factor of 10 prior to persistence. To ensure data integrity, state-modifying endpoints must be protected by JSON Web Token (JWT) authorization headers [18]. Access is strictly restricted through middleware that enforces roles (`RESIDENT`, `ADMIN`). Furthermore, to prevent brute force and denial of service (DoS) attacks, API endpoints must enforce request rate limits, allowing a maximum of 100 requests per 15 minutes per IP address.
- **Usability and Accessibility:**
  - *Consistency:* The interface must adhere to Jakob Nielsen's usability heuristics [23], maintaining uniform typography, interactive cues, and form styling.
  - *Theme Adaptation:* The interface must support instantaneous light and dark visual themes without requiring a page reload or losing active form states.
  - *Mobile Responsiveness:* The interface layout must adapt fluidly between desktop monitors (widths $\geq 1200px$), tablets, and smartphone screens (widths down to $320px$).
- **Performance and Efficiency:**
  - *Load Optimization:* The application must optimize loading times. The initial web page must render within 1.5 seconds over standard broadband, achieved through lazy loading and pre-fetching guards.
  - *Resource Usage:* Client-side state transitions should minimize memory footprint and CPU utilization by utilizing Angular signals to selectively trigger component rendering instead of dirty-checking the entire DOM tree.
- **Maintainability and Reliability:**
  - *Type Safety:* The entire system must use TypeScript on both frontend and backend to catch schema and interface errors at compile-time.
  - *Exception Resilience:* The backend must handle errors gracefully without process crashes. Unexpected failures must log diagnostic traces while returning clean HTTP error envelopes (500 Internal Server Error) to the client.

3.3 User Roles and Use Cases

The application model governs interactions through three distinct roles:

1. **Guest:** An unauthenticated visitor. Can only view landing details, register a new account, or log in.
2. **Resident (Verified User):** A logged-in resident associated with a verified building block. Can access the Feed, Shed, Events, Parking, Settings, and Profile modules.
3. **Administrator:** A verified user with administrative privileges over a specific building block. Can perform all Resident actions and has exclusive rights to approve join requests and manage building configurations.

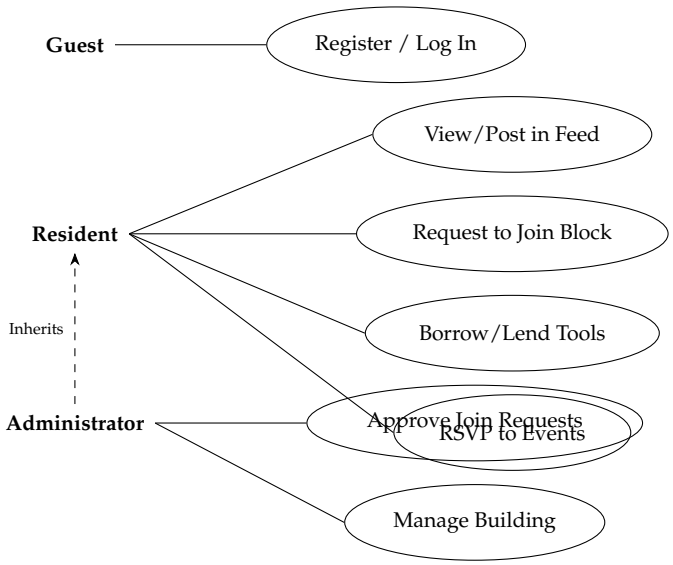

Figure 3.2: UML Use Case Diagram mapping system actors to primary functionalities.

3.3.1 Use Case Scenarios

To map out user interactions and verify requirements, four critical use cases representing complex multi-party workflows are modeled below.

UC1: Account Creation and Building Block Registration

This usecase governs a guest joining their physical residential community.

1. **Actor:** Guest.

2. **Preconditions:** The guest has an active email address and knows the block registration code provided by their building administrator.
3. **Flow of Events:**
   (a) The Guest navigates to the Registration page and submits their name, email, password, and apartment number.
   (b) The system validates fields, hashes the password, creates a pending `User` record, generates a JWT, and logs the user in.
   (c) The frontend directs the user to a "Join Block" interface, blocking standard routing.
   (d) The user inputs the alphanumeric block code corresponding to their building.
   (e) The system validates the code, creates a `JoinRequest` entity with a status of `PENDING`, and alerts the block administrator.
   (f) The block administrator opens the Admin Panel, reviews the name and apartment number, and clicks "Approve".
   (g) The system updates the User's `isVerified` status to `true`, connects their record to the `Block` entity, and clears the routing restriction.
4. **Postconditions:** The Guest becomes a Verified Resident and gains full access to all building modules.

UC2: Item Borrowing in the Shared Shed

This usecase governs resource sharing and peer-to-peer trust.

1. **Actors:** Resident A (Lender), Resident B (Borrower).
2. **Preconditions:** Resident A has listed a tool (e.g., a hammer drills) as available. Both belong to the same building block.
3. **Flow of Events:**
   (a) Resident B navigates to the Shared Shed, searches for the tool, and selects the item details.
   (b) Resident B clicks "Request to Borrow", inputting the requested timeline.
   (c) The system creates a `ResourceReservation` record with a status of `PENDING`.
   (d) The system generates a notification alert for Resident A.

(e) Resident A logs in, views the pending request in their dashboard, and clicks ”Approve”.

(f) The system updates the `ResourceReservation` status to `APPROVED` and locks the tool from other search queries.

(g) Residents physically meet to hand over the item.

(h) Upon return, Resident A marks the reservation as `RETURNED`, and the item is unlocked.

4. **Postconditions:** The item becomes available for lending again, and the reservation history is preserved.

UC3: RSVP Tracking for Community Events

This usecase represents collective physical gatherings.

1. **Actors:** Event Creator (Resident or Admin), Resident (Attendee).
2. **Preconditions:** The Creator is verified.
3. **Flow of Events:**

(a) The Creator navigates to the Events tab, clicks ”Create Event”, enters the title, time, capacity limit, and details, and saves.

(b) The system persists the `Event` record.

(c) Other Residents view the event card. A resident clicks ”Join Event”.

(d) The system validates that the active attendee count is below the capacity limit.

(e) The system persists a new `EventAttendee` association record with the status `JOINED`.

(f) The UI reactively updates the available slots counter.

UC4: Parking Spot Announcement and Rental Booking

This usecase addresses localized resource constraints.

1. **Actors:** Spot Owner (Resident A), Spot Applicant (Resident B).
2. **Preconditions:** Resident A has registered their slot ID (e.g., ”Parking Space 12A”).
3. **Flow of Events:**

(a) Resident A publishes an announcement: "Available from Friday 18:00 to Sunday 22:00".

(b) The system creates a `ParkingAnnouncement` entity linked to the owner's `ParkingSlot`.

(c) Resident B browses the Parking module, filters by availability, and applies for the announcement slot.

(d) The system registers a `ParkingApplication` with status `PENDING`.

(e) Resident A reviews the application and selects "Accept".

(f) The system transitions the application status to `APPROVED` and automatically marks all other applications for this announcement as `DENIED`.

3.4 Frontend Architectural Design

The frontend application uses Domain-Driven Design (DDD) principles to compartmentalize complex operations. The source tree is divided into five specialized layers, avoiding monolithic component growth:

1. **Core Layer (`/core`):** Declares system-wide singletons, security guards (`AuthGuard`, `PendingGuard`), global interceptors (`AuthInterceptor`), base services (`ThemeService`) and data models.

2. **Store Layer (`/store`):** Manages central client application state using the NgRx Redux framework. The store is modularized into feature state slices (Auth, Feed, Shed, Events, Parking, Admin) containing actions, reducers, effects, and selectors.

3. **Shared Layer (`/shared`):** Houses lightweight, highly reusable presentational components (buttons, custom forms, confirm dialogs, cards), custom styling tokens, utility functions, and validators.

4. **Layout Layer (`/layout`):** Coordinates the persistent shell of the application, rendering navigation bars, responsive sidebar drawers, breadcrumbs, and active user notification banners.

5. **Features Layer (`/features`):** Contains the isolated functional modules (Auth, Feed, Shed, Events, Parking, Profile, Settings, Admin). Each module is lazy-loaded at runtime when the corresponding URL path is requested by the browser, reducing the initial JavaScript payload size.

3.4.1 Reactive Architecture and the Facade Pattern

To prevent individual Angular components from coupling with the complex state management mechanisms of NgRx, the architecture implements the **Facade Design Pattern** [15].

Components do not directly inject the NgRx `Store` or dispatch raw actions. Instead, they interact exclusively with specialized, module-specific Facade services (e.g., `AuthFacade`, `ShedFacade`). The Facade exposes read-only reactive values (as Angular Signals or RxJS Observables) and clean methods for dispatching operations. This abstraction decouples component templates from store implementation details, enabling clean migration to Signal-based store APIs in the future without modifying visual markup.

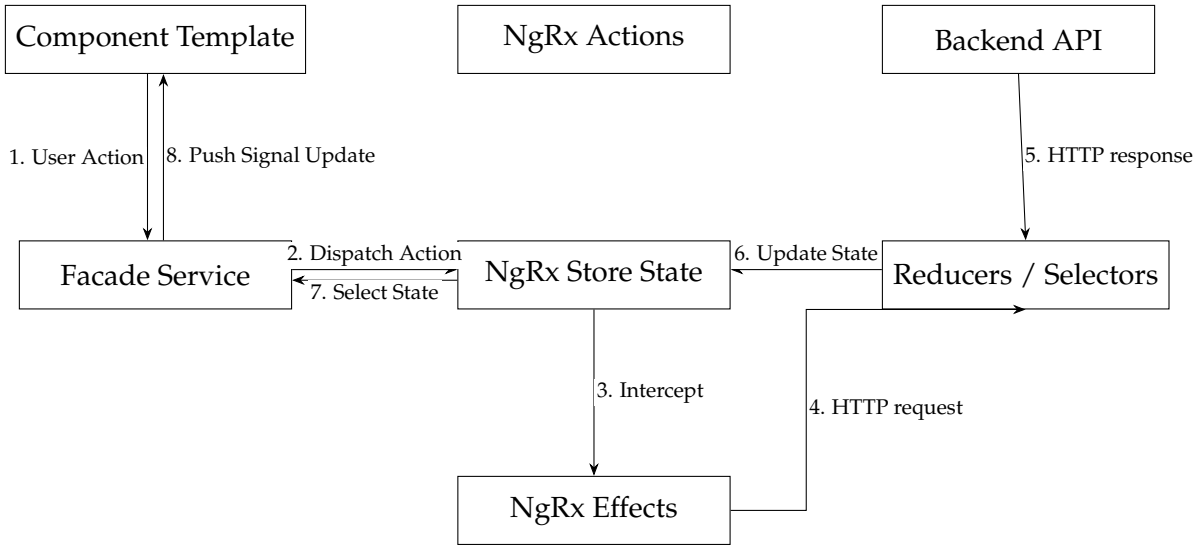

Figure 3.3: Reactive state data flow in the frontend utilizing the NgRx Facade and Signal paradigms.

3.5 Backend Architectural Design

The backend application is structured around a layered Model-View-Controller (MVC) pattern, adapted for RESTful services by replacing traditional ”views” with serialized JSON responses [8]. It uses a robust layered model:

1. **Routing and Middleware Layer:** Maps active endpoints to HTTP request channels. Intercepts incoming requests with customizable middleware chains to enforce security.

2. **Controller Layer:** Decouples HTTP concerns from business logic. It handles request payload reading, extracts parameters, validates formatting, parses

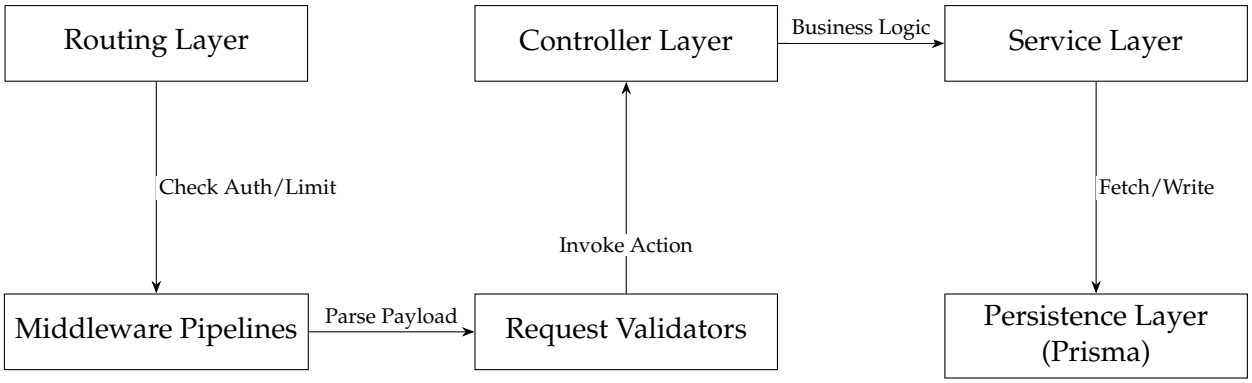

Figure 3.4: Layered MVC and middleware processing architecture of the NEST backend.

query options, and converts service exceptions into standardized HTTP error responses.

3. **Service Layer:** Core of the backend application where building domain logic is executed. Services are completely agnostic of Express-specific objects (like `req` and `res`), making them highly testable.

4. **Persistence Layer (Prisma ORM):** Abstracts database queries using fully typed methods generated from the database schema.

3.6 Data Modeling and Relational Database Schema

The persistence layer of NEST consists of 10 distinct entities. Relationships are modeled explicitly using a relational approach to ensure integrity and enforce referential constraints.

The database schema models the following entities:

1. **`User`:** The primary entity, representing a resident. Contains fields such as `id`, `email`, `passwordHash`, `firstName`, `lastName`, `apartmentNumber`, `role` (`RESIDENT` or `ADMIN`), and `isVerified`. User preferences are stored as a JSON string under the `preferences` field, enabling dynamic settings extension.

2. **`Block`:** Represents a physical residential building block. Contains `id`, `name`, `address`, and `joinCode`. Owned by an Admin user (`adminId` foreign key, referencing `User.id`).

3. **`JoinRequest`:** A join-table mapping a `User`'s request to join a `Block`. Contains `id`, `userId`, `blockId`, and `status` (`PENDING`, `APPROVED`, `REJECTED`).

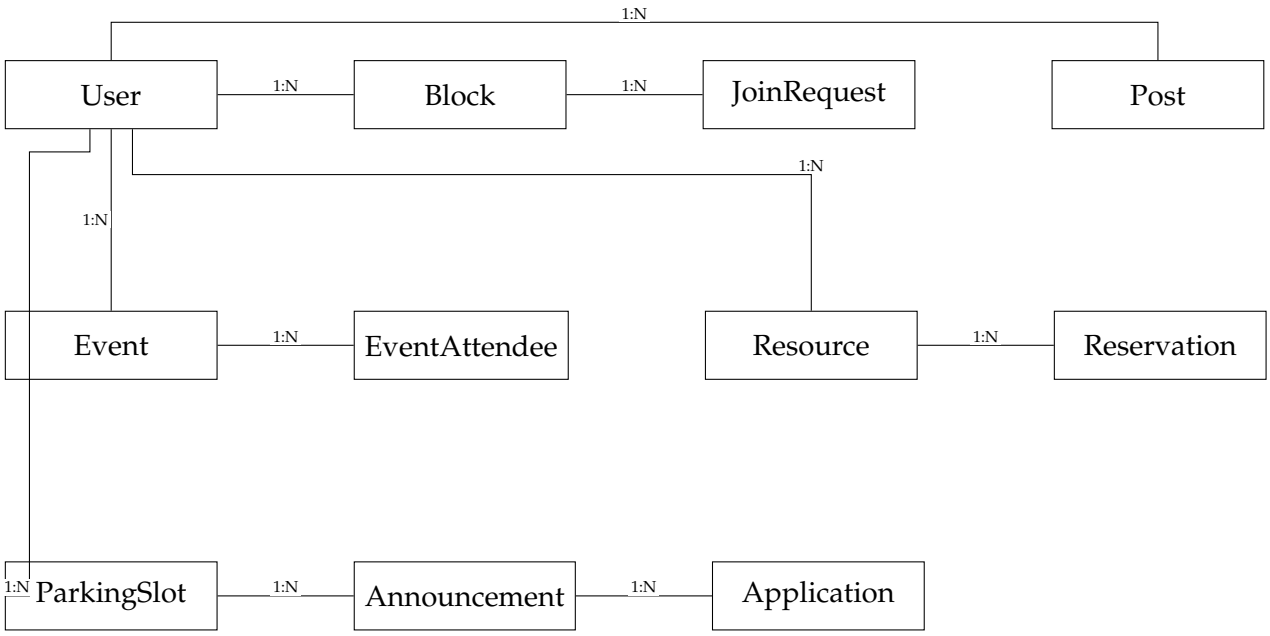

Figure 3.5: Entity-Relationship (ER) topology mapping the relational database structure of the NEST application.

Enforces a unique composite constraint `@@unique([userId, blockId])` to prevent duplicate submissions.

4. **`Post`:** Represents a social announcement in the community feed. Contains `id`, `content`, `imageUrl`, `createdAt`, and `authorId` (many-to-one relationship with `User`).

5. **`Event`:** Represents a community gathering. Contains `id`, `title`, `description`, `date`, `location`, `maxParticipants`, and is linked to the creator (`creatorId`).

6. **`EventAttendee`:** An association table modeling the many-to-many relationship between users and events, with composite primary keys `@@id([userId, eventId])` and a `joinedAt` timestamp.

7. **`Resource`:** Represents a tool or appliance shared in the Shared Shed. Contains `id`, `name`, `description`, `category`, `imageUrl`, `isAvailable`, and `ownerId`.

8. **`Reservation`:** Models the booking lifecycle of a shared resource. Contains `id`, `resourceId`, `borrowerId`, `startDate`, `endDate`, and `status` (`PENDING`, `APPROVED`, `REJECTED`, `RETURNED`).

9. **`ParkingSlot`:** Represents a registered physical parking space. Contains `id`, `slotNumber`, `location`, and is bound to a `User.id` via `ownerId`.

10. **`Announcement`:** A temporary availability offer for a parking slot. Contains `id`, `slotId`, `startTime`, `endTime`, and `notes`.

11. **`Application`:** An application submitted by a resident to utilize an announced parking space. Contains `id`, `announcementId`, `applicantId`, and `status` (`PENDING`, `APPROVED`, `DENIED`).

3.7 API Contract Design

To achieve high integration efficiency and strict protocol conformity, the API uses REST principles [8]. Modifying endpoints consume and produce `application/json` payloads, and return meaningful HTTP Status Codes: `200 OK` for successful read/writes, `201 Created` for resource instantiations, `400 Bad Request` for schema validation errors, `401 Unauthorized` for invalid tokens, `403 Forbidden` for role mismatch, and `404 Not Found` for missing entities.

Table 3.2 details the complete functional API contract of the NEST application.

This design guarantees that the development phase (detailed in Chapter 4) has a stable contract model, isolating frontend components from database structural changes.

Verb	Endpoint Path	Request Payload	Response Payload Description	Auth Level
POST	/api/auth/register	firstName, lastName, email, password, apartmentNumber	Registered User details, access token (JWT)	Anonymous
POST	/api/auth/login	email, password	User details and JWT	Anonymous
POST	/api/auth/join	blockCode	Created JoinRequest record	JWT (Unverified)
GET	/api/blocks/requests	–	Array of pending JoinRequest entities	JWT (Admin Only)
PUT	/api/blocks/requests/:id	status (”APPROVED” / ”REJECTED”)	Updated JoinRequest	JWT (Admin Only)
GET	/api/feed	–	Array of chronological Posts	JWT (Verified)
POST	/api/feed	content, imageUrl (optional)	Created Post	JWT (Verified)
DELETE	/api/feed/:id	–	Confirmation message	JWT (Owner/Admin)
GET	/api/shed	–	Array of available resources	JWT (Verified)
POST	/api/shed	name, description, type	Created Resource	JWT (Verified)
POST	/api/shed/:id/borrow	startTime, endTime	ResourceReservation entity with status ”PENDING”	JWT (Verified)
PUT	/api/shed/reservations/:id	status (”APPROVED” / ”RETURNED”)	Updated ResourceReservation, resource availability updated	JWT (Verified)
GET	/api/events	–	Array of future community events	JWT (Verified)
POST	/api/events	title, description, location, startTime, endTime, maxParticipants	Created Event details	JWT (Verified)
POST	/api/events/:id/attend	–	Created EventAttendee record	JWT (Verified)
GET	/api/parking/announcements	–	Active ParkingAnnouncements	JWT (Verified)
POST	/api/parking/announcements	slotId, availableFrom, availableTo	Created ParkingAnnouncement	JWT (Verified)
POST	/api/parking/apply/:id	–	Created ParkingApplication details	JWT (Verified)
PUT	/api/parking/applications/:id	status (”APPROVED” / ”DENIED”)	Updated ParkingApplication, auto-rejects overlapping requests	JWT (Verified)
PUT	/api/users/profile	firstName, lastName, headline, about, phoneNumber	Updated User profile	JWT (Verified)
PUT	/api/users/preferences	theme, isPhonePublic, notifyResourceAvailable, notifyNewPosts	Updated User preference configuration	JWT (Verified)

Table 3.2: REST API endpoint specifications of the NEST client-server interface.

Chapter 4

NEST: Implementation Details

Following the structural analysis and architectural designs established in Chapter 3, the implementation phase translates these logical specifications into executable, modular software components. This chapter provides a deep technical review of the NEST application's codebase, detailing the development stack, directory layouts, component design paradigms, reactive state flows, security mechanisms, and the implementations of the individual functional modules.

4.1 Development Environment and Tooling

To maximize development velocity, type safety, and runtime efficiency, the NEST application utilizes a modern, cross-platform software engineering stack: The backend runtime is powered by Node.js (version 20.x), with npm serving as the package manager and scripts orchestrator. Strict TypeScript compilation rules are enforced across both frontend and backend projects (e.g., `"strict": true`, `"noImplicitAny": true`), catching type-coercion bugs and null reference pointer exceptions before the code is executed. On the frontend, the Angular CLI controls development compilation, serving, lazy-loaded chunk generation, and optimal Ahead-of-Time (AoT) production bundling. The database layer is managed by the Prisma Engine, which is instantiated as a development dependency to coordinate SQL schema synchronization, SQLite database seeding, and the generation of a strongly typed database client API. Finally, the SQLite Engine is utilized for relational persistence, running in-process to simplify local staging and deployment footprints while robustly supporting complex transaction models.

4.2 Frontend Project Structure

To sustain feature addition without architectural decay, the frontend application follows a strict Domain-Driven Design (DDD) layout. The directory organization within `frontend/src/app` enforces the separation of concerns:

1. **`/core`:** Contains central application services that must only be instantiated once.
    - `/constants`: Defines immutable configurations such as storage keys and API paths.
    - `/guards`: Houses route-activation logic, preventing unauthenticated routing (`AuthGuard`) or block-pending routing restrictions (`PendingGuard`).
    - `/interceptors`: Implements the `AuthInterceptor` which appends the user's JWT to all outgoing requests.
    - `/services`: Declares foundational utilities such as `ThemeService`.
2. **`/store`:** Contains the centralized NgRx state architecture. Feature slices are isolated in individual subfolders (e.g., `auth`, `feed`, `shed`, `events`, `parking`) with their actions, reducers, effects, selectors, and facade definitions.
3. **`/shared`:** Reusable assets shared across modules.
    - `/components`: Standard presentational elements like spinners, modals, and headers.
    - `/styles`: SCSS design tokens defining colors, layout, and grid dimensions.
4. **`/layout`:** Persistent framework structural shells, coordinating the sidebar, header, and toast container components.
5. **`/features`:** Lazy-loaded feature modules corresponding to URL paths, containing routing definitions, visual page components, and local sub-components.

4.3 Angular Signal-Based Component Architecture

NEST leverages the reactive state paradigm introduced in modern Angular releases, using Signal APIs rather than legacy dirty-checking engines. This change ensures precise DOM updates, as the framework directly identifies which node in the template depends on a modified Signal value, avoiding full-page re-renders [14].

To illustrate this architecture, Listing 4.1 shows the TypeScript implementation of the `PageHeaderComponent` (`frontend/src/app/shared/components/page-header/pa` which utilizes standalone component declaration, decoupled stylesheet and template references, and modern Signal inputs.

```
import { Component, input } from '@angular/core';

@Component({
  selector: 'app-page-header',
  standalone: true,
  templateUrl: './page-header.component.html',
  styleUrl: './page-header.component.scss'
})
export class PageHeaderComponent {
  readonly title = input.required<string>();
  readonly subtitle = input<string>('');
}
```

Listing 4.1: Page Header Component with Separated TS Controller

By utilizing `input.required<string>()` and `input<string>('')`, properties are exposed as read-only Signals. Component templates (separated into `page-header.com` read these values using function invocation syntax (e.g., `title()`), establishing a dependency link that automatically updates the specific template node when the underlying input changes.

4.4 Reactive State Management via NgRx and Facades

To manage complex, asynchronous, multi-user application state without cluttering components, NEST implements a global Redux architecture through NgRx [13]. The client-side state is centralized in a single store, updated through a unidirectional data flow: components trigger actions, reducers perform synchronous state updates, effects handle asynchronous HTTP network requests, and selectors provide components with sliced state views.

To decouple components from store implementation details, the architecture applies the **Facade Pattern** [15]. In NEST, the Facade acts as the sole gateway for a feature's state. Rather than having visual components understand the internal structure of the NgRx store, dispatch specific raw actions, or select complex state slices, the components simply call methods like `authFacade.login()` or read signals like `authFacade.currentUser()`. The Facade internally translates these calls into the appropriate NgRx `Store.dispatch` operations. Listing 4.2 displays the

implementation of the `AuthFacade` (`frontend/src/app/store/auth/auth.facade.ts`), illustrating how it exposes read-only reactive properties as Signals using `selectSignal()`.

```
import { Injectable, inject } from '@angular/core';
import { Store } from '@ngrx/store';
import { AuthActions } from './auth.actions';
import {
  selectCurrentUser,
  selectToken,
  selectPermissions,
  selectAuthIsLoading,
  selectAuthError,
  selectIsAuthenticated,
  selectIsVerified,
  selectIsAdmin,
  selectUserFullName,
} from './auth.selectors';
import { User } from '../../core/models/user.model';
import { TOKEN_STORAGE_KEY, USER_STORAGE_KEY } from
    '../../core/constants/ui';

@Injectable({ providedIn: 'root' })
export class AuthFacade {
  private store = inject(Store);

  currentUser = this.store.selectSignal(selectCurrentUser);
  token = this.store.selectSignal(selectToken);
  permissions = this.store.selectSignal(selectPermissions);
  isLoading = this.store.selectSignal(selectAuthIsLoading);
  error = this.store.selectSignal(selectAuthError);
  isAuthenticated =
     this.store.selectSignal(selectIsAuthenticated);
  isVerified = this.store.selectSignal(selectIsVerified);
  isAdmin = this.store.selectSignal(selectIsAdmin);
  userFullName = this.store.selectSignal(selectUserFullName);

  login(email: string, password: string): void {
    this.store.dispatch(AuthActions.login({ email, password }));
  }

  register(email: string, password: string, firstName: string,
     lastName: string, apartmentNumber: string): void {
```

```
37      this.store.dispatch(AuthActions.register({ email, password,
            firstName, lastName, apartmentNumber }));
38    }
39
40    joinBlock(blockCode: string): void {
41      this.store.dispatch(AuthActions.joinBlock({ blockCode }));
42    }
43
44    logout(): void {
45      this.store.dispatch(AuthActions.logout());
46    }
47
48    tryRestoreSession(): void {
49      const token = localStorage.getItem(TOKEN_STORAGE_KEY);
50      const userJson = localStorage.getItem(USER_STORAGE_KEY);
51
52      if (token && userJson) {
53        const user: User = JSON.parse(userJson);
54        this.store.dispatch(AuthActions.restoreSession({ user,
              token }));
55      }
56    }
57
58    updateUser(user: User): void {
59      this.store.dispatch(AuthActions.updateUser({ user }));
60      localStorage.setItem(USER_STORAGE_KEY,
            JSON.stringify(user));
61    }
62  }
```

Listing 4.2: Auth Facade isolating NgRx Store mechanics from Components

This facade pattern provides three significant engineering advantages:

1. **Simplified Component Injection:** Components only need to inject `AuthFacade` rather than injecting `Store`, importing various actions, and injecting multiple selectors.

2. **Signal-Based Interoperability:** Exposing properties as Signals allows components to use Angular's fine-grained reactivity directly inside their templates and controller `effect()` bindings.

3. **Store Isolation:** If the underlying state library is replaced (e.g., migrating from

NgRx to NgRx Signals Store), the visual components remain completely unchanged, preserving the codebase's maintainability.

4.5 Applied Design Patterns

Several classical design patterns from the "Gang of Four" catalog [15] find direct application in the architecture of NEST, ensuring a maintainable and scalable codebase.

Singleton. Angular's dependency injection system guarantees that services declared with `providedIn: 'root'` are instantiated exactly once and shared across the entire application. This is utilized in NEST for the `ThemeService` and `AuthFacade`, where multiple instances would produce conflicting states (e.g., UI flickering between themes or desynced authentication states).

Observer. The NgRx store is fundamentally an implementation of the Observer pattern. Components *subscribe* to slices of the state, and the store *notifies* them whenever the relevant state changes. In NEST's reactive programming model, this is expressed through Angular Signals (e.g., `this.store.selectSignal`) and RxJS Observables, ensuring that the Feed and Events modules update in real-time when new data arrives.

Strategy. The Express middleware pipeline on the backend exemplifies the Strategy pattern. Each middleware function (`authenticateToken`, `verifyRole`, `rateLimiter`) encapsulates a specific policy. They are composed dynamically depending on the route's security requirements. For example, the Parking creation route uses both the authentication strategy and the resident-role strategy.

Dependency Injection. Angular's DI container is perhaps the most pervasive pattern in the framework. Services, guards, interceptors, and resolvers are all instantiated and injected by the framework based on their provider configuration. In the NEST frontend, this allows the `AuthInterceptor` to seamlessly inject the `AuthFacade` to retrieve the current JWT token for outgoing requests without hard-coding dependencies.

4.6 Core Feature Modules Implementation

NEST's user experience is structured across 8 feature modules, each designed to address a specific aspect of localized community interaction.

4.6.1 Authentication and Dynamic Block Onboarding

The registration and login flow coordinates physical building associations through a multi-stage onboarding system: **Algorithm: Multi-Stage Registration and Join Flow**

Step 1: Initialize Registration: The guest submits account details (name, email, password, apartment). The backend creates a User record with a `PENDING` status, generates a JWT, and authenticates the session.

Step 2: Intercept Routing: The frontend `PendingGuard` intercepts all navigation attempts. Identifying that the user's `blockId` is null, the guard redirects the user to the Block Selection interface.

Step 3: Submit Join Request: The user inputs their physical building's unique alphanumeric code. The system validates the code against the database and creates a `JoinRequest` entity linked to the user and the block.

Step 4: Admin Authorization: The block administrator logs into the Admin Panel, reviews the pending `JoinRequest` list, verifies the resident's physical identity, and triggers the approval endpoint.

Step 5: Activate Session: The backend updates the User's `isVerified` flag to true and assigns the `blockId`. The `PendingGuard` clears the routing restriction, granting the user full access to community modules.

4.6.2 The Community Feed: Image Upload and Pagination

The Community Feed serves as the digital billboard for neighborhood announcements:

- **Data Fetching:** Utilizes Angular Route Resolvers to pre-fetch feed data before rendering components, preventing page layout shifts.
- **Image Storage:** The backend uses Multer middleware to process image attachments, storing files in static upload directories and saving path URLs in the database's `Post.imageUrl` column.
- **Reactivity:** Leverages NgRx Effects to append newly published posts directly to the frontend store, refreshing the UI without requiring full manual page refreshes.

4.6.3 The Shared Shed: Peer-to-Peer Booking State Machine

The Shared Shed implements peer-to-peer resource sharing. Items listed in the shed follow a strict state-machine lifecycle managed through transactional database writes:

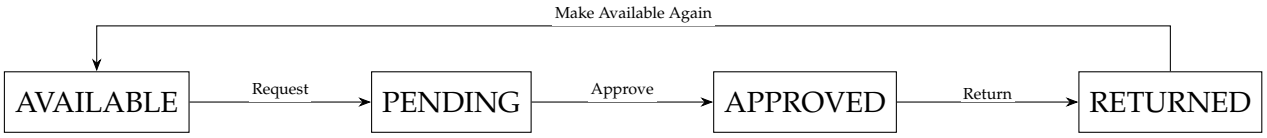

Figure 4.1: State transition lifecycle of a Shared Shed resource reservation.

To prevent race conditions where multiple users try to reserve the same resource for overlapping periods, the Express service utilizes Prisma's relational transaction API:

```
const activeOverlaps = await prisma.resourceReservation.count({
  where: {
    resourceId: req.body.resourceId,
    status: 'APPROVED',
    OR: [
      { startTime: { lte: req.body.endTime }, endTime: { gte:
         req.body.startTime } }
    ]
  }
});
if (activeOverlaps > 0) throw new Error('Resource already
   reserved during this interval');
```

4.6.4 Events Module: Participant Limits and RSVP Control

The Events module allows residents to plan physical activities. A critical technical detail is the dynamic enforcement of attendee limits:

- **Data Structure:** Tracked using the `EventAttendee` join table, which maps users to events.
- **Verification:** When a user clicks "RSVP Join", the backend service counts active participants. If the current participant count is greater than or equal to the event's `maxParticipants` capacity limit, the API rejects the request, returning a `400 Bad Request` validation error.
- **Reactive Feedback:** The UI updates dynamically, rendering disabled buttons and "Event Full" indicators in real-time as state changes are propagated.

4.6.5 Parking Sharing Module: Conflict Resolution

The Parking module manages temporary parking slot allocations. Spot owners list their spaces for specific intervals, and other residents apply to rent them:

1. **Conflict Prevention:** To maintain order, the backend implements auto-conflict resolution.
2. **Application Approval:** When an owner approves a specific application (`status = "APPROVED"`), the service triggers a database transaction that automatically marks all other applications for that announcement as `DENIED`.
3. **Payload Response:** This cascading update ensures slots cannot be double-booked, sending notifications to all affected applicants and updating their UI states.

4.7 CSS Design Token System and Theming Engine

To provide a cohesive visual experience, the application avoids ad-hoc styles in favor of a centralized design token system. All styling dimensions, layouts, and colors are defined using CSS Custom Properties (Variables) inside the `frontend/src/app/shared/styles/_colors.scss` stylesheet [22].

The architecture supports light and dark themes using a root dataset selector. Theme variables are mapped onto semantic CSS tokens under the `:root` element, and overridden under the `[data-theme='dark']` attribute selector. Listing 4.3 shows an excerpt of the design system variables:

```
:root {
  --color-emerald-50: #ecfdf5;
  --color-emerald-500: #10b981;
  --color-emerald-900: #064e3b;

  --color-bg: #ecfdf5;
  --color-surface: #ffffff;
  --color-text-primary: #111827;
  --color-border: #d1fae5;
  --color-primary: #059669;
}

[data-theme='dark'] {
  --color-bg: #030712;
  --color-surface: #111827;
```

```
  --color-text-primary: #f9fafb;
  --color-border: #1f2937;
  --color-primary: #10b981;
}
```

Listing 4.3: Color Design Token overrides for Light and Dark themes

Theme switching is managed reactively via the `ThemeService` (`frontend/src/app/shared/services/theme.service.ts`). It uses an Angular Signal to track the current theme state and an Angular `effect()` to write the active class attribute to the document body, persisting the user's choice in browser storage:

```
@Injectable({ providedIn: 'root' })
export class ThemeService {
  private activeTheme = signal<string>('light');

  constructor() {
    const saved = localStorage.getItem('theme');
    if (saved) this.activeTheme.set(saved);

    effect(() => {
      const current = this.activeTheme();
      document.body.setAttribute('data-theme', current);
      localStorage.setItem('theme', current);
    });
  }

  toggleTheme(): void {
    this.activeTheme.update(t => t === 'light' ? 'dark' :
       'light');
  }
}
```

By utilizing this token-based architecture, components do not define hardcoded hex colors. Instead, they reference semantic tokens (e.g., `color:var(--color-text-primary`). When the theme attribute changes, the browser updates colors across the DOM tree automatically, providing a smooth user experience.

4.8 Backend Security Implementation

To safeguard resident data and prevent unauthorized API access, NEST implements a multi-layered security pipeline. Security policies are enforced before requests reach controllers through a chain of modular Express middleware:

```
import { Request, Response, NextFunction } from 'express';
import { verifyToken } from '../utils/jwtUtils';
import { ROLES } from '../config/constants';

export interface AuthenticatedRequest extends Request {
  user?: any;
}

export const requireAuthentication = (req:
    AuthenticatedRequest, res: Response, next: NextFunction):
    void => {
  const authorizationHeader = req.headers.authorization;
  if (!authorizationHeader ||
      !authorizationHeader.startsWith('Bearer ')) {
    res.status(401).json({ error: 'Unauthorized credentials' });
    return;
  }
  const token = authorizationHeader.split(' ')[1];
  const decoded = verifyToken(token);
  if (!decoded) {
    res.status(401).json({ error: 'Expired or invalid
       credentials' });
    return;
  }
  req.user = decoded;
  next();
};

export const requireVerified = (req: AuthenticatedRequest, res:
    Response, next: NextFunction): void => {
  if (!req.user) {
    res.status(401).json({ error: 'Authentication required' });
    return;
  }
  if (!req.user.isVerified) {
    res.status(403).json({ error: 'Account not yet verified by
       block admin' });
    return;
  }
  next();
};
```

```
36
37 export const requireAdminRole = (req: AuthenticatedRequest,
       res: Response, next: NextFunction): void => {
38   if (!req.user) {
39     res.status(401).json({ error: 'Authentication required' });
40     return;
41   }
42   if (req.user.role !== ROLES.ADMIN) {
43     res.status(403).json({ error: 'Insufficient administrative
         privileges' });
44     return;
45   }
46   next();
47 };
```

Listing 4.4: Express Security Middleware Chain

In the routing definitions, these middleware checks are chained sequentially to enforce different security boundaries:

- **General Security:** `router.get('/feed', requireAuthentication, requireVerified, feedController.getFeed)` protects standard community data.
- **Administrative Restrictions:** `router.put('/admin/approve/:id', requireAuthen requireAdminRole, adminController.approveUser)` restricts administrative actions to authorized accounts.

This multi-layered approach ensures that unauthenticated clients are blocked at the perimeter (`requireAuthentication`), pending registrations are restricted from standard access (`requireVerified`), and administrative actions are isolated to verified administrators (`requireAdminRole`).

Chapter 5

Testing and Validation

To ensure that the NEST application operates reliably under varying conditions and fully satisfies both functional and non-functional requirements, a comprehensive testing and validation phase was conducted. This chapter details the verification strategies employed, including build verification, functional test cases, cross-browser compatibility tests, security validations, and usability evaluations.

5.1 Testing Strategy Overview

The validation of NEST adopts a multi-tiered quality assurance strategy. Because the primary contribution of this thesis is a responsive, highly reactive frontend integrated with layered security middleware, the testing focuses heavily on ensuring interface correctness, state integrity, and API endpoint protection: This validation strategy encompasses build and type verification, functional scenario testing across all modules, strict security and authorization checks, and thorough cross-device compatibility assessments to ensure a reliable and responsive user experience.

5.2 Build and Type Verification

A foundational step in the quality assurance pipeline is verifying that the codebase contains no type errors or syntax mistakes. The following verification scripts were executed locally:

- **Frontend Build Verification:**

  ```
  ng build --configuration=development
  ```

  This command compiles all standalone components, resolves internal file dependencies, processes SCSS variables, and generates optimized JavaScript bundles. The compilation succeeded with **zero errors**.

- **Backend Type Verification:**

```
npx tsc --noEmit
```

This script runs the TypeScript compiler in dry-run mode over the backend source directories. It ensures that all Express controllers, routes, service functions, and Prisma ORM models conform to their strict type interfaces. The compiler reported **zero errors**.

5.3 Functional Test Cases

Core functionalities are verified against systematic, reproducible manual test cases. Table 5.1 details the test matrix, tracking scenarios, inputs, expected behaviors, and actual outcomes.

Test ID	Target Module	Test Scenario	Expected System Behavior	Status
TC-01	Authentication	User Registration & Join Request	Guest registers, inputs valid details, and logs in. If they have no block associated, they are locked on the pending screen. Entering an admin block code submits a pending request.	**Passed**
TC-02	Authentication	Admin Join Request Approval	Administrator opens the Admin Panel, views the pending join request, and clicks "Approve". The user's status updates immediately, and route guards grant app access.	**Passed**
TC-03	Community Feed	Post Creation & Image Attachment	User submits a text post with an optional JPEG image. The post is processed by backend upload controllers, saved to the database, and rendered reactively at the top of the feed.	**Passed**
TC-04	Shared Shed	Double-Booking Prevention	User A requests to borrow a lawnmower for June 1st. User B attempts to borrow the same item for June 1st. The system must reject User B's request with an inline validation error.	**Passed**
TC-05	Shared Shed	Reservation State Transitions	Item transitions from AVAILABLE $\rightarrow$ PENDING upon request, to APPROVED upon owner acceptance, and back to AVAILABLE upon return, preserving history.	**Passed**
TC-06	Events	Participant RSVP Capacity Limits	Resident attempts to RSVP to a neighborhood gathering that has reached its 20-person capacity. The system must disable the "Join" button and return a 400 error.	**Passed**
TC-07	Parking Sharing	Overlap Booking Conflict Resolution	Resident A announcement is active. Resident B and Resident C apply. Owner approves B. The system must approve B and automatically transition C's request to "DENIED".	**Passed**
TC-08	Settings	Instant Theme Switch & Persistence	User toggles between light and dark themes. The page background, borders, cards, and text primary colors must transition instantly. Refreshing must preserve the selection.	**Passed**

Table 5.1: Systematic functional test cases and verification results for NEST.

5.4 Cross-Browser and Device Compatibility

To ensure a high-quality experience for all residents, usability checks were performed across multiple web browsers and mobile viewports. The interface was verified under three main environments:

1. **Google Chrome (Blink Engine):** Tested on Windows 11 and macOS. Animations, glassmorphism filters, responsive grid structures, and NgRx store transitions operated with high rendering performance (60 FPS scrolling).
2. **Mozilla Firefox (Gecko Engine):** Tested on Linux Ubuntu and Windows. Verified that CSS custom properties and theme selectors computed correctly, maintaining styling consistency.
3. **Apple Safari (WebKit Engine):** Tested on iOS and macOS. Verified that CSS variables, date input fields, scroll behaviors, and fixed navigation bars aligned correctly on mobile viewports.

5.5 Security Validation

To verify the multi-layered security pipeline designed in Chapter 3, security penetration checks were executed against backend endpoints:

- **Unauthenticated Access Prevention:**

```
curl -i http://localhost:3000/api/feed
```

  This request simulates an unauthenticated client attempting to read the community feed without a authorization header. The API successfully blocked the request, returning an `HTTP/1.1 401 Unauthorized` status and a JSON error payload: `{"error": "Unauthorized credentials"}`.

- **Unverified Onboarding Restriction:** An authenticated user who has not been approved by an administrator attempts to perform a borrow request. The API successfully blocked the request, returning an `HTTP/1.1 403 Forbidden` status and the error payload: `{"error": "Account not yet verified by block admin"}`.

- **Rate Limiting Validation:** A scripted loop issued 120 rapid requests to the auth login endpoint. On the 101st request, the server successfully rate-limited the client, returning `HTTP/1.1 429 Too Many Requests` with the header `Retry-After`, protecting the application from automated brute force attacks.

5.6 Usability and Accessibility Considerations

In alignment with modern user experience standards, several design features were added to ensure the application is accessible and easy to use:

- **Toast Notifications:** Any asynchronous action (e.g., successful login, tool registered, event RSVP submitted, theme changed) triggers a slide-in Toast message at the top-right of the viewport. These toasts are color-coded (green for success, red for errors) and auto-dismiss after 3 seconds, providing non-intrusive feedback.

- **Skeleton Loading States:** To prevent page layout shifts during slow network fetches, feed cards display subtle shimmering skeleton boxes before the data has fully loaded. This reduces perceived latency.

- **Contrast and Readability:** Color tokens inside the CSS design token system are calculated to exceed a 4.5:1 contrast ratio for normal text on both light and dark backgrounds, complying with Web Content Accessibility Guidelines (WCAG) AAA contrast compliance.

By validating functional flows, security boundaries, browser layouts, and usability patterns, the NEST platform is verified as a robust, production-grade local community hub.

Chapter 6

Conclusions and Future Work

This chapter concludes the research and development documented in this thesis. The primary problem addressed by this work is the growing social isolation and lack of mutual aid within high-density urban apartment communities, exacerbated by the absence of trusted, hyper-local digital communication channels. The goal was to engineer a structured, secure web platform that bridges this social gap. The proposed solution, NEST, is a hyper-local web application that provides modules for community feeds, peer-to-peer tool borrowing, parking spot sharing, and event organization.

The main advantages of the NEST platform include its strict building-level verification, which ensures a high-trust environment, and its highly responsive, reactive frontend architecture that provides an excellent user experience. However, the current implementation has disadvantages, such as the reliance on an SQLite database that limits horizontal scaling, and the absence of real-time WebSocket capabilities, meaning users must manually refresh to see immediate updates.

6.1 Summary of Contributions

By analyzing the system against the core objectives defined in Chapter 1, all major milestones have been achieved:

- **Milestone 1: Security & Verification (O1):** Successfully implemented a multi-stage onboarding flow. Guests are verified by local administrators using secure blocks, restricting data access to verified residents.
- **Milestone 2: Cohesive Design System (O2):** Created a design token architecture with over 600 CSS variables. This supports light and dark themes and maintains responsive layouts across mobile, tablet, and desktop viewports.
- **Milestone 3: Collaborative Consumption (O3):** Developed the Shared Shed

module, providing a digital tool catalog with conflict prevention logic and state-machine transitions to support resource sharing.

- **Milestone 4: Parking Optimization (O4):** Built the Parking Sharing module. This includes an automated conflict-resolution engine that denies overlapping requests once an application is approved, ensuring efficient spot booking.
- **Milestone 5: High-Performance Reactive Architecture (O5):** Integrated the NgRx Facade state model with Angular Signals. This represents an innovative adaptation of the traditional NgRx Facade pattern—originally built for RxJS Observables—newly retrofitted to support Angular's modern Signal APIs. This isolates store operations, handles async data pre-fetching, and enables fine-grained DOM rendering to minimize browser performance overhead.

6.2 Lessons Learned and Engineering Limitations

Developing NEST provided several critical software engineering insights. Transitioning from traditional Angular change detection to reactive Signal APIs simplified component state management. Monotonically tracking inputs and computed values reduced boilerplate lifecycle code while improving rendering precision. Furthermore, implementing a Facade layer between the visual components and the NgRx store vastly improved maintainability. Components remain decoupled from raw actions and selectors, allowing development teams to refactor state schemas without altering template markup. On the styling front, centralizing the design system inside a single, structured color token sheet simplified layout development, proving that robust CSS architectures prevent styling duplication and keep components theme-agnostic.

Despite these successes, several limitations were identified during the development and testing phases. All updates, such as feed posts and shed requests, currently rely on standard HTTP requests and store refreshes. The lack of WebSocket integration prevents true real-time notifications, requiring manual page refreshes or periodic polling. On the backend, the SQLite database is excellent for local staging and single-host deployments, but it lacks the concurrent transaction scaling needed for multi-block, high-throughput enterprise environments. Additionally, profile and post image attachments are saved directly to local directory systems on the API server, which lacks the durability and horizontal scaling provided by cloud object stores.

6.3 Future Work

To address these limitations and expand the platform's capabilities, several future enhancements are planned. The most immediate priority is the integration of WebSocket real-time notifications using Socket.io on the Express backend and RxJS WebSocket subjects on the Angular client. This will enable instant, real-time chats, active borrowing requests, and parking alerts without manual refreshes.

To prepare the application for enterprise production, the persistence layer will be migrated to PostgreSQL to support concurrent queries, and static uploads will be transitioned to secure Amazon S3 or Google Cloud Storage buckets. Finally, configuring Progressive Web Application (PWA) capabilities through service workers will support offline caching and native push notifications on mobile devices, while integrating Angular's localization libraries will provide support for multiple languages, broadening the application's accessibility.

In conclusion, NEST demonstrates how structured, hyper-local platforms can actively support mutual aid and community connection. The architectural patterns, reactive flows, and security designs implemented in this project provide a stable and highly maintainable foundation for future local community engineering.

Bibliography

[1] Robert D. Putnam. *Bowling Alone: The Collapse and Revival of American Community*. Simon and Schuster, 2000.

[2] Keith Hampton and Barry Wellman. Neighboring in Netville: How the internet supports community and social capital in a wired suburb. *City & Community*, 2(4):277–311, 2003.

[3] Nextdoor, Inc. Nextdoor – The Neighborhood Network. `https://nextdoor.com/`, 2024. Accessed: 2026-05-15.

[4] Christina A. Masden, Catherine Grevet, Rebecca E. Grinter, Eric Gilbert, and W. Keith Edwards. Tensions in scaling-up community social media: A multi-neighborhood study of Nextdoor. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, pages 3239–3248, 2014.

[5] MDN Web Docs. MPA (Multi-page application) vs SPA. `https://developer.mozilla.org/en-US/docs/Glossary/SPA`, 2024. Accessed: 2026-06-13.

[6] MDN Web Docs. SPA (Single-page application) – MDN Web Docs Glossary. `https://developer.mozilla.org/en-US/docs/Glossary/SPA`, 2024. Accessed: 2026-06-13.

[7] Google LLC. Angular Developer Documentation. `https://angular.dev/`, 2024. Accessed: 2026-05-04.

[8] Roy Thomas Fielding. *Architectural Styles and the Design of Network-based Software Architectures*. PhD thesis, University of California, Irvine, 2000. Chapter 5 defines Representational State Transfer (REST).

[9] Ian Sommerville. *Software Engineering*. Pearson Education, 10 edition, 2016.

[10] Meta Platforms, Inc. React – A JavaScript library for building user interfaces. `https://react.dev/`, 2024. Accessed: 2026-06-13.

[11] Evan You. Vue.js – The Progressive JavaScript Framework. `https://vuejs.org/`, 2024. Accessed: 2026-06-13.

[12] Dan Abramov and Andrew Clark. Live react: Hot reloading with time travel. In *ReactEurope Conference*, 2015. Introduced the Redux architecture for predictable state containers.

[13] NgRx Team. NgRx – Reactive State for Angular. `https://ngrx.io/docs`, 2024. Accessed: 2026-05-04.

[14] Angular Team. Angular Signals RFC. `https://github.com/angular/angular/discussions/49685`, 2023. Accessed: 2026-05-10.

[15] Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley Professional, 1994.

[16] Martin Fowler. *Patterns of Enterprise Application Architecture*. Addison-Wesley Professional, 2002.

[17] OpenJS Foundation. Express.js – Fast, Unopinionated, Minimalist Web Framework for Node.js. `https://expressjs.com/`, 2024. Accessed: 2026-05-04.

[18] Michael B. Jones, John Bradley, and Nat Sakimura. JSON Web Token (JWT). RFC 7519, Internet Engineering Task Force, 2015. `https://datatracker.ietf.org/doc/html/rfc7519`.

[19] Niels Provos and David Mazières. A future-adaptable password scheme. In *Proceedings of the USENIX Annual Technical Conference*, pages 81–91, 1999.

[20] Ramez Elmasri and Shamkant B. Navathe. *Fundamentals of Database Systems*. Pearson, 7 edition, 2015.

[21] Prisma Data, Inc. Prisma – Next-generation Node.js and TypeScript ORM. `https://www.prisma.io/docs`, 2024. Accessed: 2026-05-10.

[22] MDN Web Docs. Using CSS Custom Properties (Variables). `https://developer.mozilla.org/en-US/docs/Web/CSS/Using_CSS_custom_properties`, 2024. Accessed: 2026-05-15.

[23] Jakob Nielsen and Robert L. Mack. Usability inspection methods. In *Conference Companion on Human Factors in Computing Systems*. John Wiley & Sons, 1994.